
Ecommerce Data Warehouse Design

Team Members: Mingran Chen, Lei Jiang, Kun Lan

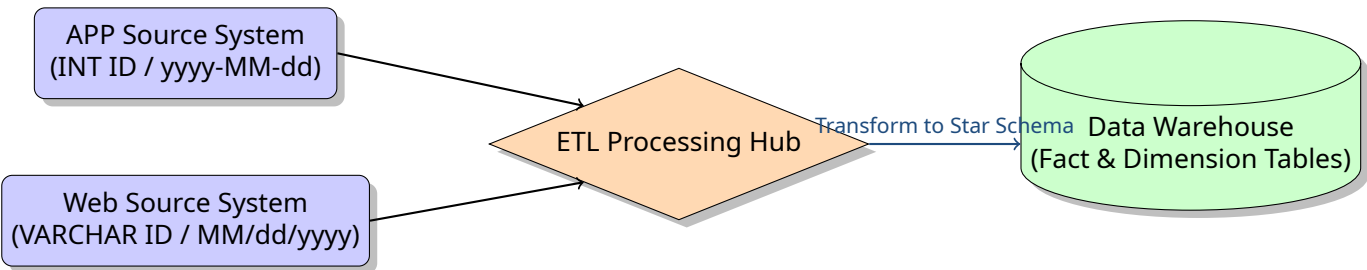
March 31, 2026

1 Project Theme and Objectives

This project focuses on the **"Integration of Heterogeneous Multi-channel Ecommerce Data and the Construction of an Analytical Data Warehouse."**

The primary goal is to transition from OLTP to OLAP using a **Star Schema** architecture. By decoupling product attributes into a dedicated dimension table, we ensure data consistency and reduce storage redundancy in fact tables.

2 System Architecture Flowchart



3 Source System Schema

This section outlines the operational database structures. The primary challenge lies in the heterogeneity between the App and Web systems.

3.1 App Source System (ecommerce_source_app)

The mobile application uses a structured schema with integer-based primary keys and standard date formats.

```
1  -- Standard Order Table in App System
2  [users]
3  (user_id, name, email, phone, city, register_date, created_at, updated_at)
4  [products]
5  (product_id, name, description, category, price, cost, brand, stock_qty,
6  is_active, created_at, updated_at)
[orders]
```

```

7 (order_id, user_id, order_date, total_amount, status, created_at, updated_at)
8 [order_items]
9 (item_id, order_id, product_id, quantity, unit_price, line_total, created_at)
10 [product_reviews]
11 (review_id, product_id, user_id, rating, comment, review_date, created_at)

```

Listing 1: App Source: Primary Transaction Tables

3.2 Web Source System (ecommerce_source_web)

The web portal presents data with a different format, requiring significant ETL processing.

```

1  -- Web Orders use String IDs and VARCHAR dates
2
3  [orders]
4  (order_no, user_id, order_date, total_amount, status, created_at, updated_at)
5  [order_items]
6  (item_id, order_no, product_id, quantity, unit_price, line_total, created_at)
7  [product_reviews]
8  (review_id, product_id, user_id, rating, comment, review_date, created_at)

```

Listing 2: Web Source: Specific Data Characteristics

4 Database Design: Star Schema

In this optimized version, we implement a Dimension Table to manage product metadata extracted from the source systems.

4.1 Product Dimension Table (dim_products)

```

1 CREATE TABLE dim_products (
2     product_key INT PRIMARY KEY AUTO_INCREMENT, -- Surrogate Key
3     product_id INT NOT NULL,                    -- Source System Business ID
4     product_name VARCHAR(200) NOT NULL,
5     category VARCHAR(50),
6     brand VARCHAR(50),
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
8     UNIQUE KEY uniq_product (product_id)
9 ) ENGINE=InnoDB;

```

Listing 3: DDL for Product Dimension Table

4.2 Fact Tables (Linking to Dimension)

The fact tables now manage the product_key as a foreign key to the dimension table.

```

1 CREATE TABLE fact_sales_by_product_time (
2     id INT PRIMARY KEY AUTO_INCREMENT,
3     product_key INT NOT NULL, -- FK to dim_products
4     year INT NOT NULL,
5     month INT NOT NULL,
6     day INT,
7     total_quantity INT DEFAULT 0,
8     total_sales_amount DECIMAL(15,2) DEFAULT 0,
9     FOREIGN KEY (product_key) REFERENCES dim_products(product_key),
10    INDEX idx_time (year, month, day)

```

```
11 ) ENGINE=InnoDB;
```

Listing 4: Sales Fact Table with Product Key

5 OLAP Operations and Join Query Examples

Queries now utilize JOIN operations between fact and dimension tables to support business intelligence.

5.1 Roll up (with Dimension Join)

```
1 SELECT
2     d.category, f.year, f.month,
3     SUM(f.total_quantity) AS monthly_qty,
4     SUM(f.total_sales_amount) AS monthly_sales
5 FROM fact_sales_by_product_time f
6 JOIN dim_products d ON f.product_key = d.product_key
7 GROUP BY d.category, f.year, f.month
8 ORDER BY f.year, f.month, d.category;
```

Listing 5: Roll up: Monthly sales aggregated by Product Category

5.2 Drill down and Slice

```
1 SELECT
2     d.category, f.year, f.month, f.day,
3     f.total_quantity, f.total_sales_amount
4 FROM fact_sales_by_product_time f
5 JOIN dim_products d ON f.product_key = d.product_key
6 WHERE d.category = 'Electronics' AND f.year = 2024 AND f.month = 3
7 ORDER BY f.day;
```

Listing 6: Drill down: Analyzing Electronics daily sales for March 2024

```
1 SELECT
2     f.year, f.month, f.day, f.total_sales_amount
3 FROM fact_sales_by_product_time f
4 JOIN dim_products d ON f.product_key = d.product_key
5 WHERE d.category = 'Clothing'
6 ORDER BY f.year, f.month, f.day;
```

Listing 7: Slice: Clothing category trend analysis

5.3 Dice and Pivot

```
1 SELECT
2     d.category, f.year, f.month,
3     SUM(f.total_quantity) AS qty,
4     SUM(f.total_sales_amount) AS sales
5 FROM fact_sales_by_product_time f
6 JOIN dim_products d ON f.product_key = d.product_key
7 WHERE f.year = 2024 AND f.month IN (3, 4)
8     AND d.category IN ('Electronics', 'Clothing')
9 GROUP BY d.category, f.year, f.month
10 ORDER BY f.month, d.category;
```

Listing 8: Dice: Comparing Electronics and Clothing for March-April 2024

6 Data Mining and Analysis Benefits

By adopting the Star Schema:

- **Data Consistency:** Product name or category updates only need to be applied in `dim_products`.
- **Storage Efficiency:** Fact tables store 4-byte integers (`product_key`) instead of long strings.
- **Scalability:** New dimensions can be added without modifying existing fact table logic.